

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1101

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture.
(BL3)

Assessment Tool Weightage: 40%

QUESTIONS: Any 4

Total Marks:40

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

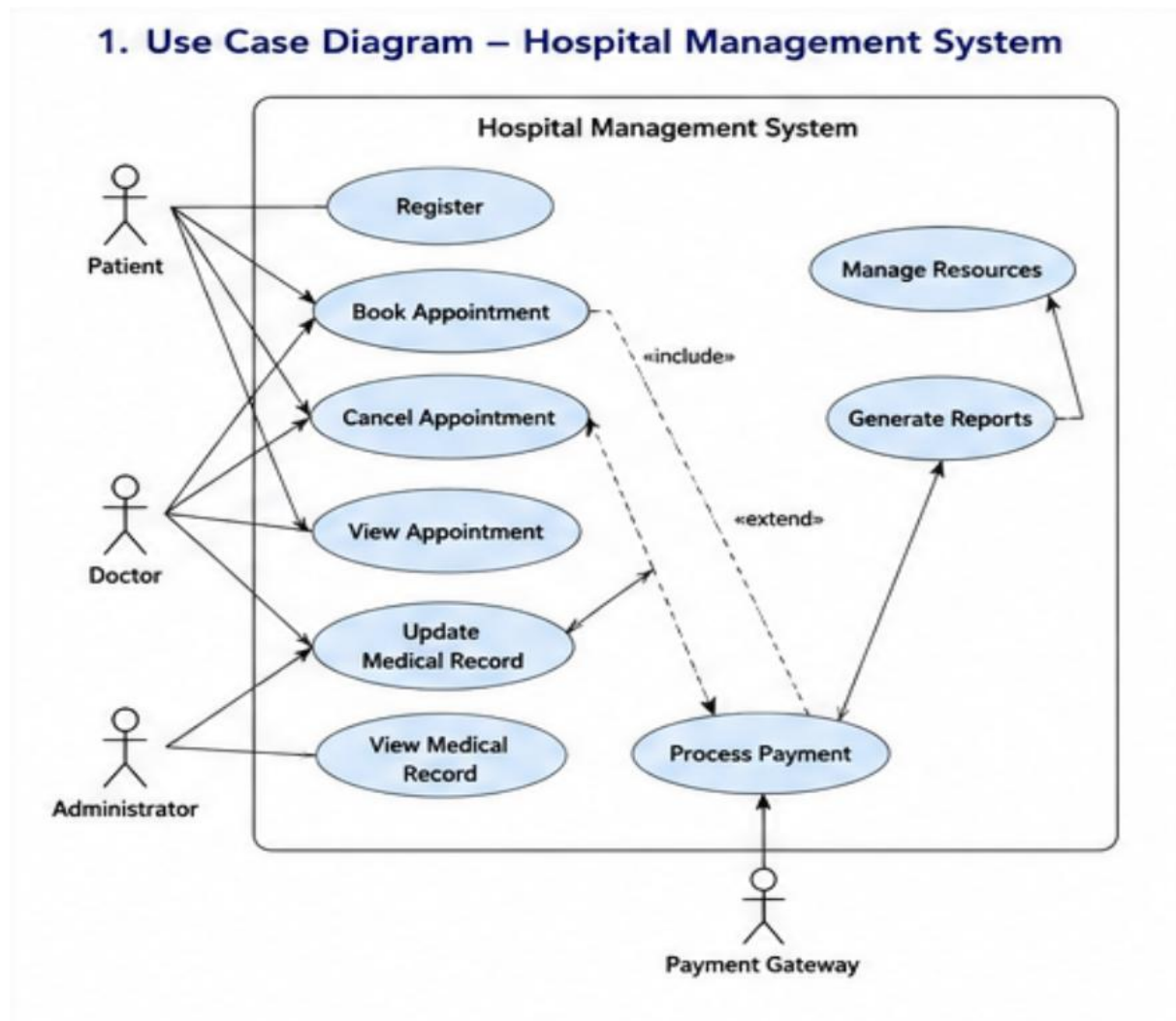
I Mathusri P-192311363 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Mathusri P

ANALYTICAL PROBLEM SOLVING

Question 1 (10 Marks)

Use Case Diagram Problem – Hospital Management System



Problem Analysis

The given Hospital Management System allows patients to book appointments, doctors to update medical records, and administrators to manage hospital resources. However, the requirements are incomplete because only three major functionalities are mentioned. Important system operations such as patient registration, authentication, payment processing, notification, and report generation are missing. Without identifying all actors and their interactions, the Use Case Diagram will not represent the complete functionality of the system.

Therefore, the system requirements must be analyzed before constructing the UML diagram.

a) Identify all actors involved in the system

After analyzing the requirements, the following actors are identified:

Actor	Responsibility
Patient	Register, login, search doctors, book and cancel appointments, make payment, view medical records
Doctor	View appointments, update medical records, prescribe treatment
Administrator	Manage users, hospital resources, doctors, reports and appointments
Payment Gateway	Process online payments securely

b) List the primary use cases

The major use cases identified from the system are:

- Patient Registration
- Login
- Search Doctor
- Book Appointment
- Cancel Appointment
- View Appointment Status
- Update Medical Record
- View Medical Record
- Process Payment
- Generate Reports
- Manage Hospital Resources
- Send Notifications

These use cases cover all important hospital operations and satisfy the functional requirements.

c) Define relationships

Include Relationship

The **Book Appointment** use case always requires:

- Search Doctor
- Check Availability
- Process Payment

Therefore, these are modeled using <<include>> relationships.

Extend Relationship

The **Cancel Appointment** use case may require a refund only if payment has already been completed. Similarly, booking confirmation is generated only after successful payment.

Therefore:

- Cancel Appointment <<extend>> Refund Payment
- Book Appointment <<extend>> Send Confirmation

Generalization

Patient, Doctor, and Administrator are specialized users of the Hospital Management System because all of them perform common operations such as Login while having additional responsibilities.

d) Analytical Solution

The optimized Use Case Diagram should contain:

- Five actors
- Twelve major use cases
- Include relationships for mandatory operations
- Extend relationships for optional operations
- Generalization to eliminate duplicate functionality

This design ensures that every system function is assigned to the appropriate actor without redundancy.

Analysis

The original problem statement does not specify payment processing, notifications, or authentication. If these functions are ignored, the Hospital Management System becomes incomplete because appointments cannot be confirmed securely and administrators cannot monitor system activities effectively.

Using **include** relationships reduces duplication by ensuring common functions such as payment processing are reused. The **extend** relationship models optional behavior, making the diagram flexible and realistic. Generalization minimizes repeated actor definitions and improves maintainability.

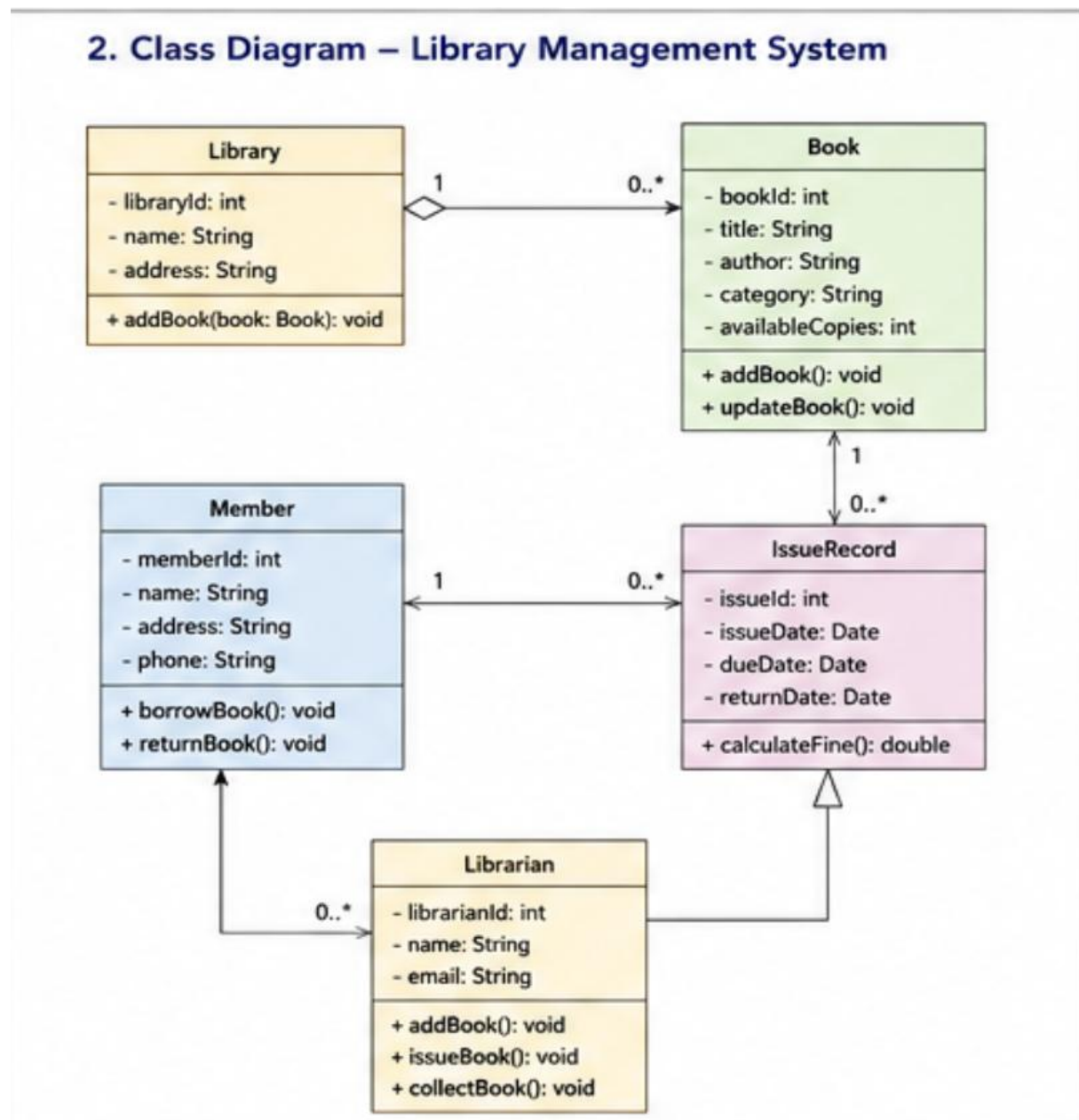
The resulting UML model is modular, scalable, and suitable for future enhancements such as laboratory management, pharmacy services, or online consultation.

Conclusion

After analyzing the incomplete requirements, the missing actors, use cases, and UML relationships were identified and incorporated into the design. The final Use Case Diagram accurately represents the complete functionality of the Hospital Management System and provides a strong foundation for software development.

Question 2 (10 Marks)

Class Diagram Problem – Library Management System



Problem Analysis

The given Library Management System consists of four major entities: Book, Member, Librarian, and IssueRecord. However, the problem states that the relationships among these entities are not properly defined. Without proper attributes, methods, relationships, and multiplicity, the system cannot accurately represent how books are issued, returned, or managed.

To design an effective Class Diagram, the classes must be analyzed, appropriate attributes and methods should be identified, and the relationships between classes should be established.

a) Identify Classes with Attributes and Methods

After analyzing the system requirements, the following classes are required.

Class 1: Book

Attributes

- BookID
- Title
- Author
- Publisher
- ISBN
- Category

Methods

- addBook()
- searchBook()
- updateBook()
- removeBook()
- checkAvailability()

Analysis

The Book class stores complete information about every book available in the library. It also provides operations to maintain and search the book database.

Class 2: Member

Attributes

- MemberID
- MemberName
- Address
- PhoneNumber

- Email
- MembershipType

Methods

- borrowBook()
- returnBook()
- reserveBook()
- viewBorrowHistory()

Analysis

The Member class represents students or faculty members who borrow books. It maintains personal details and borrowing activities.

Class 3: Librarian

Attributes

- LibrarianID
- Name
- Email
- PhoneNumber
- Department

Methods

- issueBook()
- receiveBook()
- addBook()
- generateReport()

Analysis

The Librarian manages all library operations, including issuing books, receiving returned books, maintaining inventory, and generating reports.

Class 4: IssueRecord

Attributes

- IssueID
- IssueDate
- DueDate
- ReturnDate
- FineAmount

Methods

- calculateFine()
- generateIssueSlip()
- updateReturnStatus()

Analysis

The IssueRecord class stores complete information about book issue transactions and helps track due dates and fines.

b) Define Relationships

After identifying the classes, the relationships among them are analyzed.

Association Relationship

Association represents communication between independent objects.

Relationship

- A Member borrows a Book.
- A Librarian issues a Book.
- A Librarian manages IssueRecords.

Justification

Association is suitable because all these classes interact with one another but can exist independently.

Aggregation Relationship

Aggregation represents a "Has-A" relationship where objects can exist independently.

Relationship

The Library contains:

- Books
- Members
- Librarians

Even if the library database changes, these objects can still exist independently.

Justification

Aggregation provides flexibility because removing the library does not permanently remove books or members.

Inheritance Relationship

Some classes share common information.

A new superclass called Person can be introduced.

Person

Common attributes:

- Name
- Address
- Phone Number
- Email
- Child classes:
- Member
- Librarian

Justification

Inheritance removes duplicate attributes and improves code reuse.

c) Specify Multiplicity Between Classes

After analyzing the interactions, the following multiplicities are identified.

Member — Book

One Member can borrow many Books.

One Book can be borrowed by one Member at a time.

Multiplicity:

Member (1) ----- (0..*) Book

Member — IssueRecord

One Member can have many Issue Records.

Each Issue Record belongs to one Member.

Multiplicity:

Member (1) ----- (0..*) IssueRecord

Book — IssueRecord

One Book may appear in many Issue Records over time.

Each Issue Record refers to only one Book.

Multiplicity:

Book (1) ----- (0..*) IssueRecord

Librarian — IssueRecord

One Librarian manages many Issue Records.

Each Issue Record is handled by one Librarian.

Multiplicity:

Librarian (1) ----- (0..*) IssueRecord

d) Analytical Solution

To construct an optimized Class Diagram, the following design decisions are taken.

Classes

Book

Member

Librarian

IssueRecord

Person (Superclass)

Relationships

- Association between Member and Book
- Association between Librarian and IssueRecord
- Aggregation between Library and Book/Member/Librarian
- Inheritance from Person to Member and Librarian

Multiplicity

- One Member → Many Books
- One Member → Many Issue Records
- One Book → Many Issue Records
- One Librarian → Many Issue Records

This design accurately represents the real-world library system while minimizing redundancy.

System Analysis

The original problem statement lacks proper class relationships and object interactions. By analyzing the requirements, additional relationships such as aggregation and inheritance were introduced.

Inheritance eliminates repeated attributes by introducing the Person superclass.

Aggregation improves modularity by organizing books, members, and librarians under the Library entity.

Association represents the actual interaction between books, members, librarians, and issue records.

Multiplicity ensures that the relationships reflect real-world library operations accurately.

This analytical design produces a modular, reusable, scalable, and maintainable system.

Advantages

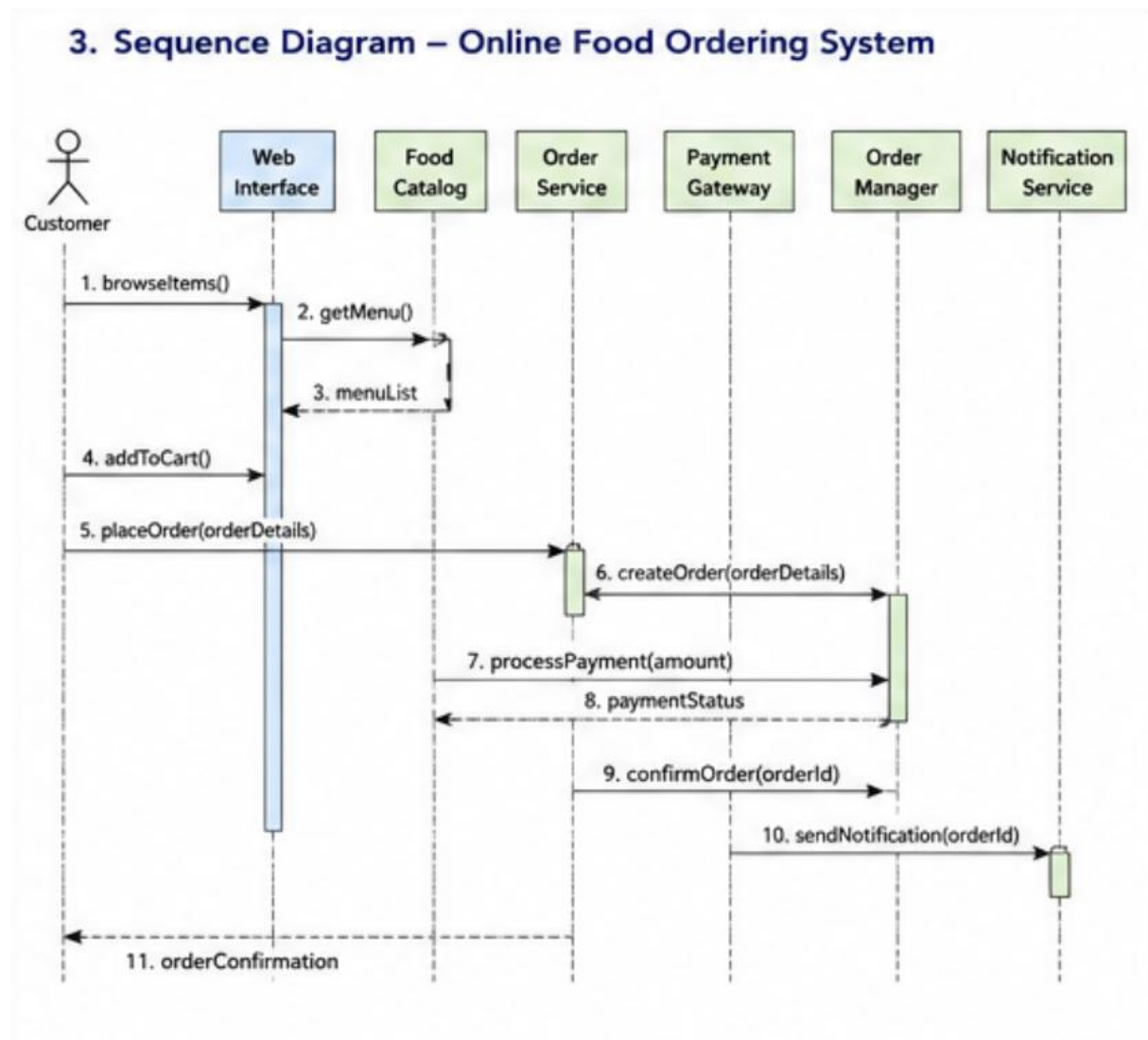
- Clearly defines class responsibilities.
- Eliminates duplicate attributes using inheritance.
- Improves code reusability.
- Represents real-world relationships accurately.
- Supports future expansion.
- Simplifies software maintenance.
- Improves documentation.
- Makes implementation easier.

Conclusion

After analyzing the incomplete Library Management System, appropriate classes, attributes, methods, relationships, and multiplicities were identified. The optimized Class Diagram provides a complete object-oriented representation of the system and ensures better modularity, scalability, and maintainability. The design satisfies all functional requirements while following UML and object-oriented design principles, making it suitable for efficient software development.

Question 3 (10 Marks)

Sequence Diagram Problem – Online Food Ordering System



Problem Analysis

The given Online Food Ordering System allows a customer to select food items, place an order, make payment, and receive order confirmation. However, the interaction among the system components is not clearly defined. The problem does not specify how information flows between the customer, ordering system, restaurant, payment gateway, and notification service.

To solve this problem, the sequence of communication between all objects must be analyzed. Every message exchanged during the ordering process should be identified so that the Sequence Diagram accurately represents the system behavior.

a) Identify Objects Involved in the Process

After analyzing the system, the following objects are identified.

1. Customer

The customer interacts with the system to search food items, place orders, and make payments.

2. Food Ordering System (User Interface)

Acts as the main interface between the customer and the backend services.

3. Restaurant Database

Stores menu details, food availability, and pricing information.

4. Order Management System

Processes customer orders and verifies item availability.

5. Payment Gateway

Processes online payments securely and returns the payment status.

6. Notification Service

Sends order confirmation messages through SMS or email.

b) Define Message Flow Between Objects

The interaction between objects takes place in the following sequence.

Step 1

The **Customer** sends a request to search food items.

Step 2

The **Food Ordering System** requests menu details from the **Restaurant Database**.

Step 3

The **Restaurant Database** returns the available food items to the system.

Step 4

The **Food Ordering System** displays the available menu to the customer.

Step 5

The **Customer** selects food items and places an order.

Step 6

The **Food Ordering System** forwards the order details to the **Order Management System**.

Step 7

The **Order Management System** verifies item availability with the **Restaurant Database**.

Step 8

The **Restaurant Database** confirms item availability.

Step 9

The **Order Management System** requests payment through the **Payment Gateway**.

Step 10

The **Customer** completes the payment.

Step 11

The **Payment Gateway** verifies the payment and returns the payment status.

Step 12

If payment is successful, the **Order Management System** confirms the order.

Step 13

The **Notification Service** sends the order confirmation to the customer.

c) Represent Activation and Lifelines

Lifelines

The lifelines represent the existence of each object during the interaction.

The lifelines are:

- Customer
- Food Ordering System
- Restaurant Database
- Order Management System

- Payment Gateway
- Notification Service

Activation Bars

Activation bars indicate the period during which an object is actively processing a request.

Examples:

- The Food Ordering System becomes active while retrieving menu information.
- The Restaurant Database becomes active while searching food items.
- The Payment Gateway becomes active while processing payment.
- The Notification Service becomes active while sending confirmation.

This clearly represents the execution sequence of each object.

d) Analytical Solution

After analyzing the incomplete interaction flow, the optimized Sequence Diagram should contain:

Objects

- Customer
- Food Ordering System
- Restaurant Database
- Order Management System
- Payment Gateway
- Notification Service

Messages

- Search Food
- Display Menu
- Select Food
- Place Order
- Verify Availability
- Process Payment
- Payment Success
- Confirm Order
- Send Notification

Alternative Conditions

If food is unavailable:

- Display "Item Not Available."
- Allow customer to choose another item.

If payment fails:

- Cancel the order.
- Display payment failure message.
- Allow payment retry.

If payment succeeds:

- Confirm the order.
- Generate Order ID.
- Notify the customer.

These conditions make the Sequence Diagram more realistic and complete.

System Analysis

The original problem provides only the basic ordering process without showing the communication between different system components. By analyzing the workflow, additional objects such as the Restaurant Database, Payment Gateway, Order Management System, and Notification Service were identified.

The sequence of messages ensures that every action follows the correct order. Payment is processed only after food availability is confirmed, and order confirmation is sent only after successful payment. This prevents invalid orders and improves system reliability.

Including activation bars and lifelines provides a clear understanding of object interactions and execution flow, making the system easier to design, test, and maintain.

Advantages

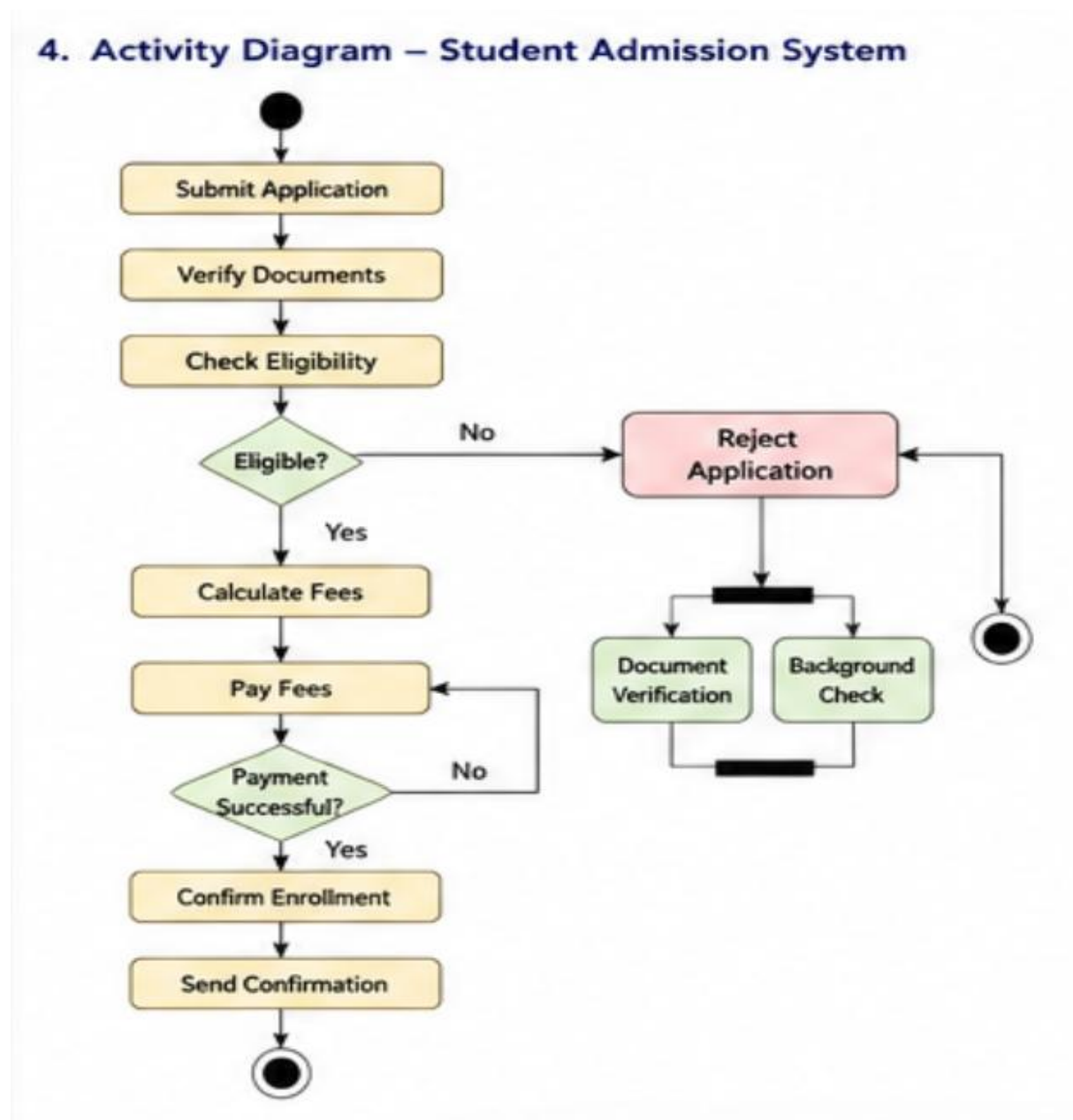
- Clearly represents communication between objects.
- Displays the chronological order of operations.
- Identifies missing interactions in the system.
- Improves system documentation.
- Simplifies debugging and testing.
- Ensures accurate order processing.
- Supports future enhancements.
- Improves software reliability and maintainability.

Conclusion

After analyzing the Online Food Ordering System, all required objects, message sequences, lifelines, and activation periods were identified. The optimized Sequence Diagram accurately represents the interaction between the customer, ordering system, restaurant database, payment gateway, and notification service. This analytical design improves system clarity, reduces communication errors, and provides a strong foundation for implementing a reliable and efficient Online Food Ordering System.

Question 4 (10 Marks)

Activity Diagram Problem – Student Admission System



Problem Analysis

The given Student Admission System includes the activities of application submission, document verification, fee payment, and enrollment confirmation. However, the workflow does not clearly define the decision-making process for application approval or rejection. It also does not indicate where multiple activities can occur simultaneously. Without proper decisions and parallel processing, the admission process becomes incomplete and difficult to understand.

To solve this problem, the workflow must be analyzed, all activities identified, decision points included, and parallel processes represented in the Activity Diagram.

a) Identify All Activities in the Workflow

After analyzing the admission process, the following activities are identified:

1. Student Registration
2. Fill Admission Application
3. Upload Required Documents
4. Submit Application
5. Verify Documents
6. Verify Eligibility Criteria
7. Approve or Reject Application
8. Generate Admission Offer
9. Pay Admission Fee
10. Verify Fee Payment
11. Generate Student ID
12. Confirm Enrollment
13. Update Student Database
14. Send Admission Confirmation

These activities represent the complete admission process from registration to final enrollment.

b) Add Decision Nodes for Approval/Rejection

Decision nodes are required whenever the system must choose between two possible outcomes.

Decision 1 – Document Verification

After verifying the uploaded documents:

- If the documents are complete and valid, the application proceeds to eligibility verification.
- If the documents are incomplete or incorrect, the application is rejected or returned for correction.

Decision 2 – Eligibility Verification

After checking eligibility:

- If the student satisfies the admission criteria, the admission offer is generated.
- If the student does not satisfy the criteria, the application is rejected.

Decision 3 – Fee Payment

After payment processing:

- If payment is successful, enrollment continues.
- If payment fails, the student is asked to retry the payment.

These decision nodes ensure that only eligible students complete the admission process.

c) Include Parallel Processing Where Applicable

Some activities can be performed simultaneously to reduce processing time.

Parallel Activity 1

After the application is submitted:

- Document Verification
- Eligibility Verification

These activities can be carried out by different departments simultaneously.

Parallel Activity 2

After successful fee payment:

- Generate Student ID
- Update Student Database

Both tasks can be executed at the same time before sending the admission confirmation.

Parallel processing improves system efficiency and reduces the admission processing time.

d) Analytical Solution

Based on the analysis, the optimized Activity Diagram should contain:

Activities

- Registration
- Application Submission
- Document Upload
- Document Verification
- Eligibility Verification
- Approval Decision
- Fee Payment
- Payment Verification
- Student ID Generation
- Database Update
- Enrollment Confirmation

Decision Nodes

- Documents Valid?
- Student Eligible?
- Payment Successful?

Parallel Processing

- Document Verification and Eligibility Verification
- Student ID Generation and Database Update

Final Activity

- Send Admission Confirmation
- End Admission Process

This design represents a complete and optimized admission workflow.

System Analysis

The original workflow was incomplete because it did not include important decision points or parallel activities. By analyzing the process, additional validation steps such as document verification, eligibility checking, and payment verification were introduced.

Decision nodes prevent invalid or incomplete applications from moving to the next stage. Parallel processing reduces delays by allowing independent tasks to execute simultaneously. This results in faster admission processing and better utilization of resources.

The optimized Activity Diagram provides a clear representation of the entire admission workflow and minimizes processing errors.

Advantages

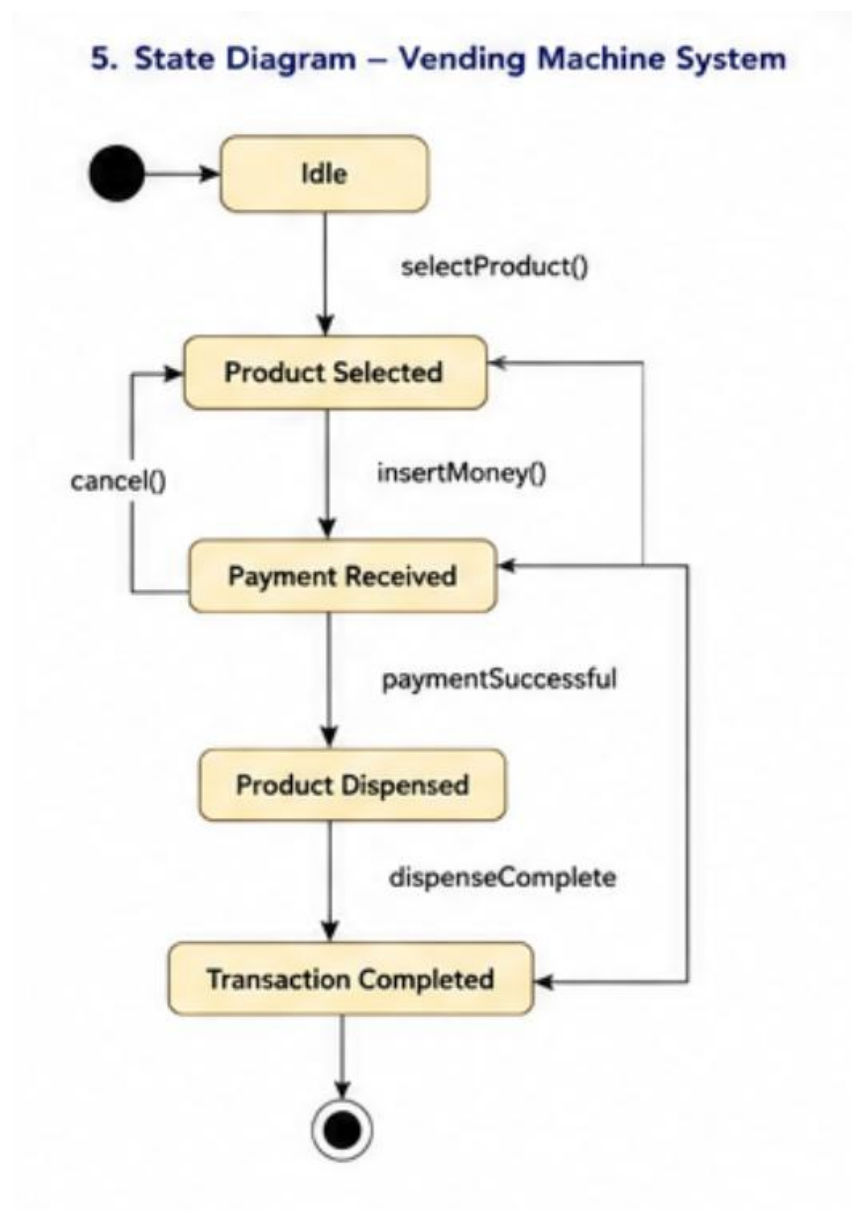
- Clearly represents the admission workflow.
- Includes all major admission activities.
- Uses decision nodes for approval and rejection.
- Supports parallel processing to improve efficiency.
- Reduces admission processing time.
- Improves workflow clarity.
- Simplifies system implementation and testing.
- Enhances overall system reliability.

Conclusion

After analyzing the Student Admission System, all required activities, decision nodes, and parallel processes were identified and incorporated into the workflow. The optimized Activity Diagram accurately represents the complete admission process from application submission to enrollment confirmation. This analytical approach improves workflow efficiency, reduces processing delays, and ensures that only eligible students successfully complete the admission process.

Question 5 (10 Marks)

State Diagram Problem – Vending Machine System



Problem Analysis

The given Vending Machine System operates through different states such as **Idle**, **Product Selected**, **Payment Received**, **Product Dispensed**, and **Transaction Completed**. However, the transitions between these states are not clearly defined. The problem does not specify the events that trigger state changes, error conditions, or how the system returns to its initial state after completing a transaction.

To develop a complete State Transition Diagram, all possible states, transition events, initial state, and final state must be analyzed.

a) Identify All Possible States

After analyzing the vending machine operation, the following states are identified:

1. Initial State

This is the starting state of the vending machine before any customer interaction begins.

2. Idle State

The machine waits for the customer to select a product. During this state, the machine displays available products and prices.

3. Product Selected State

The customer selects a product. The machine checks whether the selected product is available in stock.

4. Payment Received State

The customer inserts cash or makes an online payment. The machine verifies whether the payment is sufficient.

5. Product Dispensed State

Once payment is verified successfully, the machine dispenses the selected product.

6. Transaction Completed State

The machine updates inventory, prints or displays the receipt if required, returns any balance amount, and completes the transaction.

7. Final State

The transaction ends and the vending machine returns to the Idle State, waiting for the next customer.

b) Define Events Triggering Transitions

The transition from one state to another occurs because of specific events.

Transition 1

Initial State → Idle State

Event: Machine Power ON

Transition 2

Idle State → Product Selected State

Event: Customer selects a product.

Transition 3

Product Selected State → Payment Received State

Event: Customer confirms the selected product and initiates payment.

Transition 4

Payment Received State → Product Dispensed State

Event: Payment is successfully verified.

Transition 5

Product Dispensed State → Transaction Completed State

Event: Product is delivered to the customer.

Transition 6

Transaction Completed State → Idle State

Event: Transaction finishes successfully and the machine becomes ready for the next customer.

Alternative Events

If the selected product is unavailable:

- Display "Out of Stock."
- Return to Product Selection.

If payment is insufficient:

- Request additional payment.
- Remain in Payment Received State.

If payment fails:

- Cancel the transaction.
- Refund the amount if necessary.
- Return to Idle State.

These additional transitions make the system more practical and reliable.

c) Include Initial and Final States

Initial State

The system begins from the **Initial State**, where the vending machine is switched on and initialized.

Final State

After the transaction is completed, the machine enters the **Final State** and immediately returns to the **Idle State** to serve the next customer.

Including the initial and final states clearly represents the complete lifecycle of one vending machine transaction.

d) Analytical Solution

After analyzing the vending machine workflow, the optimized State Diagram should contain the following states:

States

- Initial State
- Idle
- Product Selected
- Payment Received
- Product Dispensed
- Transaction Completed
- Final State

Events

- Machine Power ON
- Product Selected
- Payment Initiated
- Payment Verified
- Product Dispensed
- Transaction Completed
- Return to Idle

Exceptional Conditions

- Product Out of Stock
- Insufficient Payment
- Payment Failure
- Transaction Cancelled

Including these exceptional conditions ensures that the vending machine can handle both normal and abnormal situations efficiently.

System Analysis

The original problem statement identifies only the major operating states but does not define how the system moves between them. By analyzing the workflow, appropriate transition events and alternative scenarios have been introduced.

The optimized State Diagram ensures that every state has a valid entry condition and exit condition. Error handling such as insufficient payment, payment failure, and product unavailability improves the robustness of the system.

The inclusion of initial and final states makes the system behavior complete and easy to understand. This approach also simplifies implementation, testing, and future enhancements.

Advantages

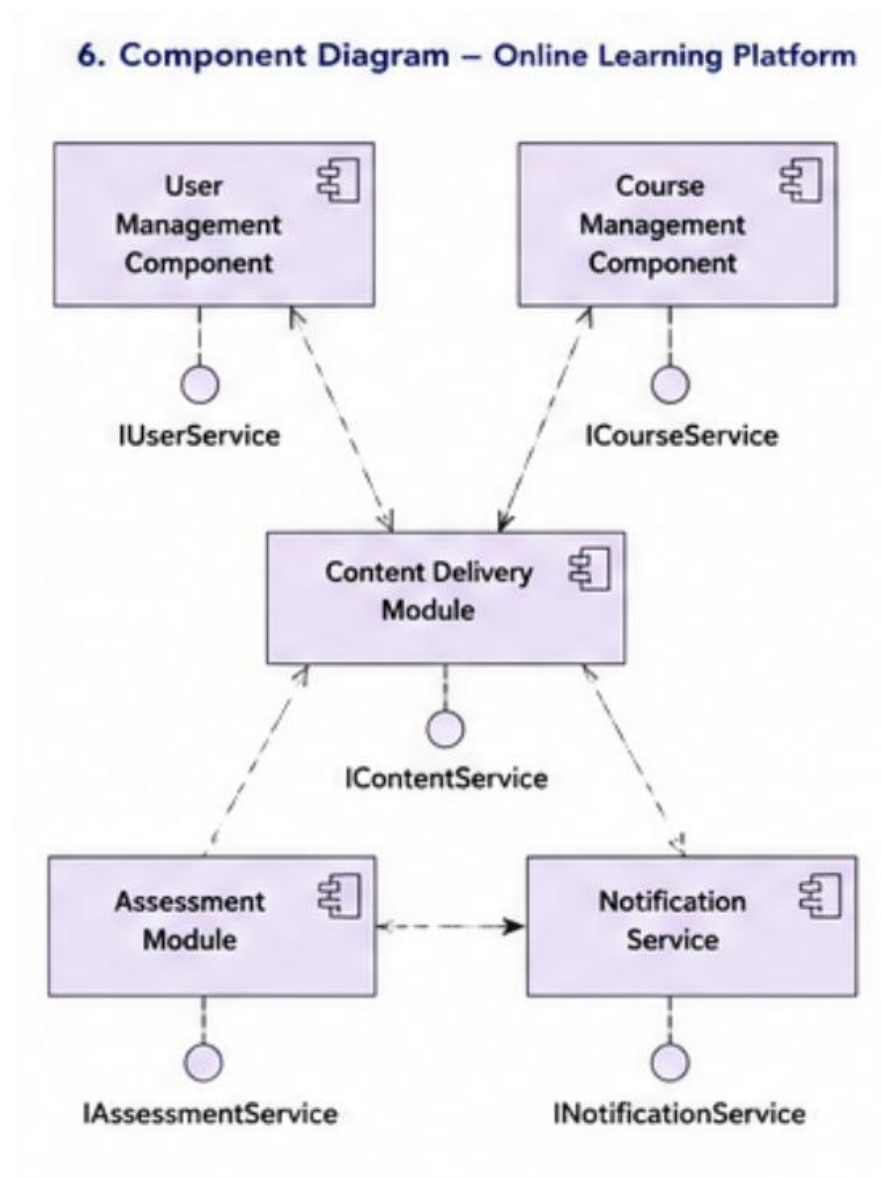
- Clearly represents system behavior.
- Defines valid state transitions.
- Handles exceptional situations effectively.
- Improves system reliability.
- Reduces operational errors.
- Simplifies software testing.
- Makes the system easier to maintain.
- Supports future enhancements and new payment methods.

Conclusion

After analyzing the Vending Machine System, all operational states, transition events, initial state, final state, and exceptional conditions were identified. The optimized State Transition Diagram accurately represents the complete behavior of the vending machine from startup to transaction completion. This analytical design improves system reliability, enhances user experience, and provides a structured model for software implementation and maintenance.

Question 6 (10 Marks)

Component Diagram Problem – Online Learning Platform



Problem Analysis

The given Online Learning Platform consists of modules such as **User Management**, **Course Management**, **Assessment Module**, **Content Delivery Module**, and **Notification Service**. However, the architecture does not clearly define how these modules communicate with each other or how they depend on one another. Without proper interfaces and dependencies, the software architecture becomes difficult to understand, maintain, and expand.

To solve this problem, the major components, their interfaces, dependencies, and interactions must be analyzed before constructing the Component Diagram.

a) Identify Major System Components

After analyzing the system requirements, the following major software components are identified.

1. User Management Component

This component manages all user-related activities.

Functions:

- User Registration
- User Login
- Profile Management
- Authentication
- Role Management (Student, Instructor, Admin)

2. Course Management Component

This component manages all learning materials and courses.

Functions:

- Create Course
- Update Course
- Delete Course
- Student Enrollment
- Course Progress Tracking

3. Assessment Module

This component manages examinations and evaluations.

Functions:

- Create Quiz
- Conduct Online Tests
- Submit Assignments
- Evaluate Answers
- Publish Results

4. Content Delivery Module

This module delivers learning resources to students.

Functions:

- Video Streaming
- PDF Notes
- Course Materials

- Recorded Lectures
- Download Resources

5. Notification Service

This component keeps users informed.

Functions:

- Course Announcements
- Assignment Reminders
- Examination Notifications
- Result Notifications
- Email and SMS Alerts

6. Database Component

Stores all application data.

Stores:

- User Information
- Course Details
- Assessment Records
- Learning Materials
- Student Progress

b) Define Interfaces Between Components

Each component communicates with other modules through well-defined interfaces.

User Management Interface

Provides authentication and user verification to all other modules.

Course Management Interface

Shares course details with:

- Content Delivery Module
- Assessment Module

Assessment Interface

Provides assessment data to:

- Notification Service
- Database

Content Delivery Interface

Supplies learning materials to students through the User Interface.

Notification Interface

Communicates with:

- User Management
- Course Management
- Assessment Module

to send alerts and updates.

Database Interface

Allows all components to store and retrieve information securely.

c) Show Dependencies Among Modules

After analyzing the software architecture, the following dependencies are identified.

User Management → Database

The User Management module depends on the database for storing and retrieving user information.

Course Management → Database

Course details are stored and retrieved from the database.

Assessment Module → Course Management

The Assessment Module depends on Course Management because assessments are created for specific courses.

Content Delivery Module → Course Management

Learning materials are delivered according to the selected course.

Notification Service → Assessment Module

Notifications are generated after quizzes, assignments, and examinations.

Notification Service → Course Management

Announcements regarding new courses or updates are sent through this dependency.

All Components → Database

Every module depends on the database for storing and accessing information.

d) Analytical Solution

After analyzing the Online Learning Platform, the optimized Component Diagram should include the following components:

Major Components

- User Management
- Course Management
- Assessment Module
- Content Delivery Module
- Notification Service
- Database

Interfaces

- Authentication Interface
- Course Interface
- Assessment Interface
- Content Interface
- Notification Interface
- Database Interface

Dependencies

- User Management → Database
- Course Management → Database
- Assessment Module → Course Management
- Assessment Module → Database
- Content Delivery Module → Course Management
- Notification Service → Assessment Module
- Notification Service → Course Management
- All Components → Database

This architecture ensures proper communication among software modules while maintaining modularity and low coupling.

System Analysis

The original problem statement identifies only the main modules without explaining their communication or dependencies. After analyzing the system, interfaces were introduced to enable standardized communication between components.

Dependencies were identified to show how each component relies on other modules. For example, the Assessment Module depends on Course Management because assessments cannot

exist without courses. Similarly, the Notification Service depends on both Course Management and the Assessment Module to generate meaningful notifications.

The optimized Component Diagram separates responsibilities among independent modules. This modular architecture improves software scalability, maintainability, and flexibility while reducing system complexity.

Advantages

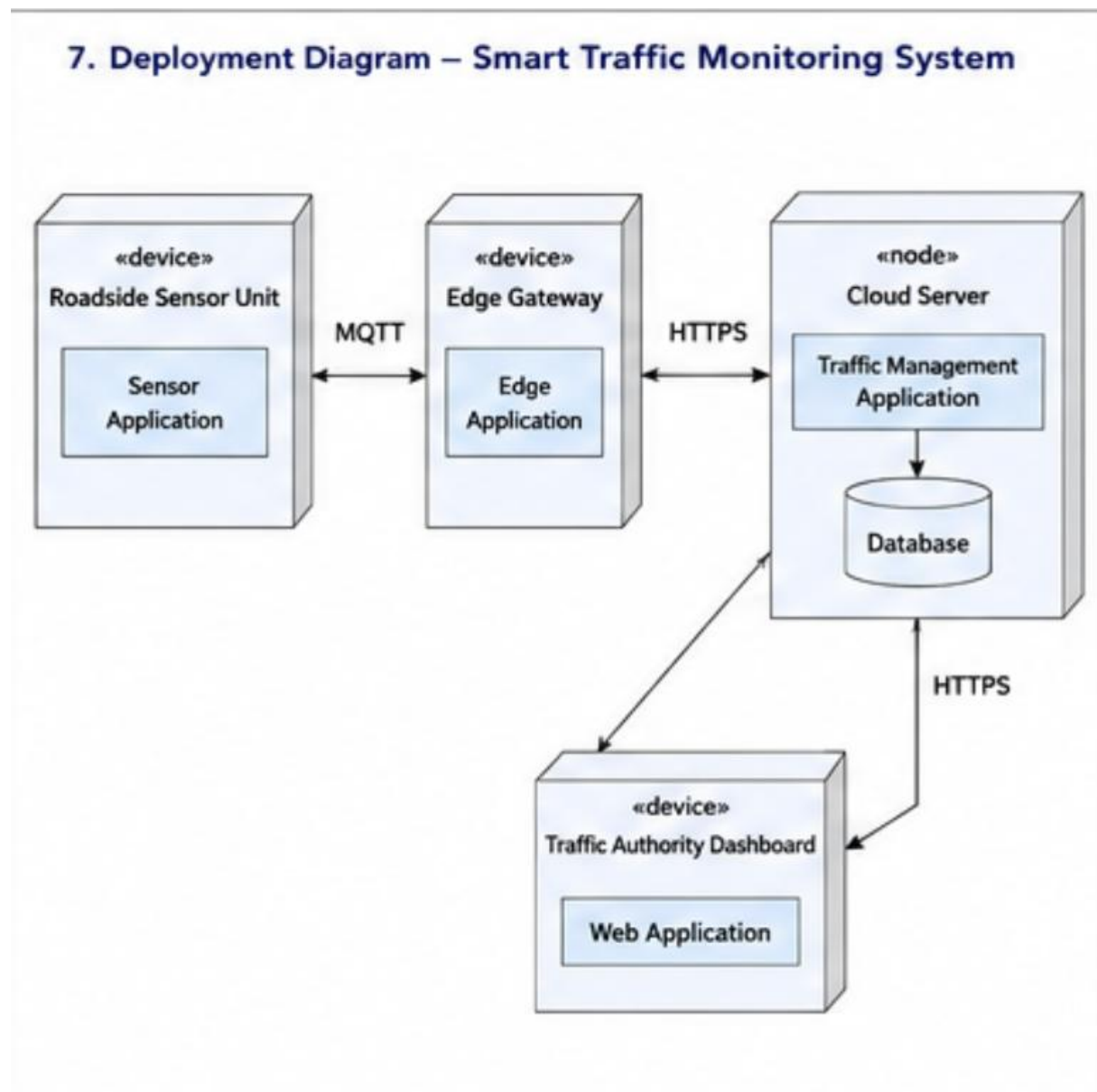
- Clearly defines software architecture.
- Separates system into independent modules.
- Improves modularity and maintainability.
- Reduces coupling between components.
- Supports future expansion.
- Simplifies testing and debugging.
- Enhances software scalability.
- Improves overall system performance and reliability.

Conclusion

After analyzing the Online Learning Platform, all major components, interfaces, and dependencies were identified and organized into a structured software architecture. The optimized Component Diagram clearly represents how different modules communicate while maintaining independence and modularity. This analytical design improves system scalability, simplifies maintenance, and provides a strong architectural foundation for developing a secure, efficient, and reliable Online Learning Platform.

Question 7 (10 Marks)

Deployment Diagram Problem – Smart Traffic Monitoring System



Problem Analysis

The given Smart Traffic Monitoring System consists of **traffic cameras, sensors, a central server, cloud storage, and monitoring stations**. However, the deployment of hardware and software components is not clearly defined. The communication between devices, server, cloud, and monitoring center is also incomplete. Without proper deployment, the system architecture cannot effectively support real-time traffic monitoring and decision-making.

To solve this problem, the physical devices (nodes), deployed software components, and communication links between them must be analyzed.

a) Identify Hardware Nodes

After analyzing the system, the following hardware nodes are identified.

1. Traffic Camera Node

This node captures live images and videos of road traffic.

Functions:

- Capture traffic images
- Record videos
- Detect vehicle movement

2. Traffic Sensor Node

This node collects traffic-related data.

Functions:

- Vehicle counting
- Speed detection
- Traffic density measurement
- Signal status monitoring

3. Central Application Server

This server processes data received from cameras and sensors.

Functions:

- Data Processing
- Traffic Analysis
- Incident Detection
- Report Generation

4. Cloud Storage Server

This node stores traffic information for future analysis.

Functions:

- Store images and videos
- Store traffic reports
- Backup historical data

5. Monitoring Station

Used by traffic control officers to monitor and manage traffic.

Functions:

- View live traffic
- Receive alerts
- Manage traffic signals
- Generate reports

b) Allocate Software Components to Nodes

Each hardware node contains specific software components.

Traffic Camera Node

- Camera Control Software
- Image Capture Module

Traffic Sensor Node

- Sensor Monitoring Software
- Data Collection Module

Central Server

- Traffic Analysis System
- AI Processing Module
- Database Management System
- Alert Generation Module

Cloud Server

- Cloud Database
- Backup System
- Data Analytics Module

Monitoring Station

- Traffic Dashboard
- Alert Notification System
- Reporting Module

c) Define Communication Links

The nodes communicate through secure network connections.

Traffic Camera → Central Server

Transfers live traffic images and videos.

Traffic Sensors → Central Server

Sends vehicle count, speed, and traffic density data.

Central Server → Cloud Storage

Stores processed traffic information and backups.

Central Server → Monitoring Station

Sends real-time traffic updates, alerts, and reports.

Monitoring Station → Central Server

Allows traffic officers to send traffic control commands and manage the system.

These communication links ensure smooth and continuous data exchange.

d) Analytical Solution

After analyzing the Smart Traffic Monitoring System, the optimized Deployment Diagram should include:

Hardware Nodes

- Traffic Camera
- Traffic Sensor
- Central Server
- Cloud Storage
- Monitoring Station

Software Components

- Camera Control Software
- Sensor Monitoring Software
- Traffic Analysis Module
- Database Management System
- Cloud Backup System
- Monitoring Dashboard

Communication Links

- Traffic Camera ↔ Central Server
- Traffic Sensor ↔ Central Server
- Central Server ↔ Cloud Storage
- Central Server ↔ Monitoring Station

This deployment architecture supports real-time traffic monitoring and efficient system management.

System Analysis

The original problem statement identifies the main devices but does not explain how software components are deployed or how the devices communicate. By analyzing the system, suitable hardware nodes, software modules, and communication links were identified.

The Central Server acts as the core processing unit, collecting data from cameras and sensors. Cloud Storage provides secure storage and backup of traffic data, while the Monitoring Station enables traffic officers to monitor conditions and respond quickly to incidents.

This deployment architecture ensures reliable communication, efficient processing, and better scalability for future system expansion.

Advantages

- Clearly defines hardware and software deployment.
- Supports real-time traffic monitoring.
- Enables efficient communication between devices.
- Improves system scalability.
- Provides secure cloud storage and backup.
- Simplifies maintenance and troubleshooting.
- Enhances system reliability and performance.
- Supports future upgrades and expansion.

Conclusion

After analyzing the Smart Traffic Monitoring System, all hardware nodes, software components, and communication links were identified and organized into an optimized Deployment Diagram. The proposed deployment architecture ensures efficient data collection, processing, storage, and monitoring. This analytical design improves system performance, reliability, scalability, and supports effective real-time traffic management, making it suitable for modern intelligent transportation systems.

RUBRICS FOR CALCULATION

ANALYTICAL PROBLEM SOLVING

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation